SIMATS ENGINEERING

CSA1023 – SOFTWARE ENGINEERING

ASSIGNMENT 1

CO1: Apply agile methodologies and project management techniques such as scale and lean to effectively manage software projects using industry-standard tools like Jira.

Student Name	B. Charan Kumar Reddy	**Register Number**	192472195
Department / Section	BE- CSE-Ai	**Academic Year / Semester**	2025-2026
Faculty Name	V. Karpagam	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	02/11/2026	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

s. No	Task Title	Max	Marks Awarded	Remarks
1	Algorithm Design and Grading Comparison	4		
2	Pipeline Design and Operation	5		
3	Image Representation Analysis	4		
4	Connectivity Analysis	4		
5	Image Acquisition Analysis	5		
6	Sampling Analysis	5		
7	Quantization Analysis	4		
8	Image Formation Analysis	5		

9	Implementation and Performance Evaluation	6		
10	Front-End Adaptation Challenge	8		
	TOTAL	**50**		

OVERALL REMARKS:

Student Signature	**Faculty Signature**	**Program Director**

SOFTWARE ENGINEERING
Assignment 1

SIMATS ENGINEERING

CSA10 – SOFTWARE ENGINEERING

ASSIGNMENT 1

CO1: *Apply agile methodologies and project management techniques such as scale and lean to effectively manage software projects using industry-standard tools like Jira.*

Assignment Title: Design, Simulate and Validate an Agile Delivery Strategy for the ITSM Platform

Nature of Assignment: Individual

Duration: One week

Total Marks: 50

Submission Date: 02/11/2026

Scenario — Agile Delivery Planning for the ITSM Platform

An enterprise plans an integrated ITSM platform to reduce delayed incident response, repeated handovers and unclear ownership. Service desk analysts prioritize intake and SLA visibility, SREs prioritize telemetry correlation, and managers require auditable approvals and developer issue links. An eight-member team must select a feasible six-week release despite uncertain capacity, limited stakeholder time and delayed integration access. Create and validate an Agile delivery strategy connecting these priorities to a justified backlog, team responsibilities, Lean flow controls and sprint goals. Revise the strategy when business units, staffing and coordination needs change, while protecting a complete incident-resolution journey.

Planning conditions for the representative release are stated below.

Observation	Scenario planning condition
Operational scope	Incident intake, triage, telemetry, approvals, problem investigation and DevOps links
Users	Administrator, SRE, analyst, CAB reviewer, manager and employee
Reference data	Synthetic incident records and controlled telemetry events

Integration constraint	External issue tracker may be unavailable; preserve local incident records and retry safely
Validation need	Verify priority, status, SLA, authorization, persistence and audit behaviour

The organization proposes an AI-assisted IT Service Management and Incident Resolution Platform. Its intended modules are an operations dashboard, assisted triage and diagnostics, incident lifecycle management, infrastructure telemetry, a service request catalog, change approval, problem investigation and a DevOps integration hub. The system should connect an alert or report to a tracked incident, authorized investigation, a justified change, deployment verification and closure.

An eight-member cross-functional team plans an initial release within six weeks using three two-week sprints. There is no established velocity. Test access to the external issue-tracking interface begins in week three. Service desk and change-review stakeholders can attend clarification and review meetings for two hours per week. These are assignment planning assumptions.

Principal users are enterprise administrators, SRE or incident commanders, service desk analysts, change advisory board reviewers, department managers and standard employees. They need different permissions. Analysts require useful diagnostic suggestions; managers require auditable approvals; engineers require traceable links to development issues and releases. Automation must preserve human responsibility for prioritization, approval and operational changes.

Define workload, service-level policies and acceptance thresholds as explicit planning assumptions. Dashboard values, diagnostic scores and screenshots are illustrative evidence of displayed behaviour, not proof of production accuracy, performance, live monitoring or secure authorization. Verify claims through repeatable checks. Distinguish deterministic keyword rules from trained machine learning and label simulations and proposed extensions accurately.

Design Challenge:

Management requests all eight modules in the initial release. Service desk analysts prioritize incident intake, assignment and SLA visibility; SREs prioritize telemetry-to-incident correlation; managers prioritize service and change approvals. Integration access and limited review time constrain delivery. A broad release may leave every workflow incomplete, while a narrow dashboard-only release may fail to improve incident resolution.

Design, simulate and validate an Agile project delivery strategy using a project-management tool of your choice that produces a coherent and achievable first release under these conditions.

Develop alternative delivery approaches and create a justified strategy connecting backlog priorities to the identified operational problems. Establish team responsibilities, capacity

assumptions, dependency sequencing, stakeholder review arrangements and Lean practices for controlling waiting, rework and work in progress.

Investigate at least **two suitable project-management tools**, such as Jira, Trello, Azure DevOps or GitHub Projects. Select one and justify its suitability for your strategy, considering planning, workflow management, collaboration, reporting, accessibility and cost. Explain any limitations and how you will address them.

Before simulation, predict the principal delivery bottleneck and define measurable criteria for judging the strategy. Use the resulting evidence to revise your plan and defend the features retained, deferred or resequenced. Your final design must explain **how the team will deliver, why the selected release should improve the service process, and how the chosen tool supports the strategy**.

Task 1 – Investigate the Problem and Define Delivery Outcomes

1.1 Investigation questions

1. Where do incidents lose the most time today: intake, triage, assignment, handover or approval waiting? (analysts, SRE)
2. Which single workflow must work end to end for the first release to be considered useful? (all roles)
3. Which SLA, priority and approval rules must be fixed before development, and who owns them? (service desk lead, CAB reviewer)
4. What proof do managers and CAB reviewers need that an approval was legitimate and auditable? (manager, CAB)
5. What happens to work when the external issue tracker is unavailable, and what link to developers is essential? (SRE, development team)
6. What delivery constraints (capacity, stakeholder time, integration access) most threaten a six-week release? (team, product owner)

1.2 Stakeholder evidence (SIMULATED)

Stakeholder (simulated)	**Simulated finding**	**Design implication**
Employee	Reports arrive by email/phone; no ticket number or status updates; people re-report the same issue.	Single intake form, incident ID and visible status.
Service desk analyst	Spends time re-keying details and guessing priority; SLA breaches noticed only after the deadline; 3–4 handovers per incident.	Priority matrix, SLA clock, assignment queue with ownership.
SRE / incident commander	Alerts and incidents live in separate tools; correlating takes 20–30 minutes manually.	Telemetry events linked to incidents by a clear rule.
CAB reviewer / manager	Approvals happen in chat and email; cannot show who approved what and when.	Role-restricted approval with immutable audit trail.
Development team	Receives incident details by copy-paste; links to issues/releases are lost.	Issue link with safe retry when tracker is down.

1.3 Problem statement, product goal, scope

Problem statement. Incident response is delayed because reports, alerts, approvals and developer work are tracked in disconnected places, causing repeated handovers, unclear ownership and no reliable evidence of who decided what. An eight-member team has six weeks, no velocity history, delayed integration access (week 3) and only 2 hours per week of stakeholder time.

Product goal. In the first release, let an employee report an issue and let the organisation take it through one complete, auditable path – report → tracked incident with priority and SLA → authorised investigation with linked telemetry → justified change approval → developer-issue link and deployment verification → closure – using synthetic data.

In scope (Release 1)	Out of scope / deferred
Role-based access for six roles	
Incident intake, priority, SLA clock, assignment, lifecycle	Service request catalog (only approval is kept)
Analyst queue with SLA visibility	Full problem-investigation module (a link field only)
Synthetic telemetry ingestion and one correlation rule	Full operations dashboard analytics
Keyword-rule diagnostic suggestions (if capacity allows)	Machine-learning diagnostics (PROPOSED extension)
Change request + CAB approval + manager approval	Live monitoring integration and real production data
Dev-issue link with local persistence and safe retry	Mobile app, SSO, multi-language
Deployment verification and closure	

1.4 Alternative release scopes considered

Option	Description	Assessment
A – Broad	All eight modules, thin.	Every workflow incomplete; high integration risk; reject.
B – Dashboard-only	Dashboard plus read-only views.	Looks good but does not improve resolution; reject.
C – Thin vertical slice (chosen)	One complete incident-resolution journey across five modules, minimal depth.	Delivers measurable improvement, testable end to end, fits capacity.

1.5 Feasibility observations

- Technical (ASSUMPTION): web app (React front end, REST API, relational database), synthetic data, keyword-rule engine, and an issue-tracker adapter behind an interface with a mock service so work is not blocked until week 3. Feasible for the team.
- Operational: stakeholder time (2 h/week) is the scarcest resource, so decisions (SLA matrix, role matrix) must be batched and made early. Integration access arrives in week 3, so integration work sits in Sprint 2.
- Financial (ASSUMPTION): the team is about 16 users after scaling. Jira Free covers up to 10 users, so a paid Standard plan (roughly USD 7–9 per user per month; verify current pricing) would be needed for ~2 months; estimated tool cost is below USD 300. Cloud hosting uses free/low-cost tiers with synthetic data.

1.6 Planning assumptions and measurable success criteria

Item	Detail
Workload (ASSUMPTION)	500 synthetic incident records, 2,000 synthetic telemetry events, 6 user roles, 12 end-to-end test scenarios.
SLA policy (ASSUMPTION)	P1: respond 15 min / resolve 4 h; P2: 30 min / 8 h; P3: 4 h / 3 days; P4: 1 day / 5 days. Priority = impact × urgency matrix.

Item	Detail
Team (ASSUMPTION)	8 members: Product Owner/BA, Scrum Master/delivery lead, 3 backend, 1 frontend, 1 DevOps, 1 QA. Three 2-week sprints.

ID	Measurable delivery-success criterion	Target
SC1	Sprint predictability: completed ÷ planned story points per sprint	≥ 80% by Sprint 3 (rolling)
SC2	End-to-end journey (report → closure) passes on synthetic scenarios	12 of 12 scenarios pass
SC3	Flow: median cycle time of stories; total blocked time per sprint	≤ 5 working days; ≤ 4 blocked days per team per sprint
SC4	Quality: unauthorised actions allowed in 40-case role matrix; audited state changes	0 allowed; 100% audited
SC5	Release readiness: all Must stories meet Definition of Done; open Sev-1/2 defects	100% Must done; 0 Sev-1/2 open

Task 2 – Design and Justify the Project Management Approach

2.1 Two delivery approaches compared

Aspect	Approach A: Scrum + XP	Approach B: Lean flow (Kanban)
How work is selected	Product Owner orders backlog; team forecasts a sprint at planning.	Pull from a prioritised queue whenever capacity frees up; no fixed batch.
Coordination	Daily stand-up, sprint goal, pair programming, CI, test-first for SLA/priority logic.	Visual board, explicit policies, WIP limits, replenishment meeting, daily flow review.
Review	Sprint review (stakeholders), retrospective each 2 weeks.	Service-delivery review and flow metrics (cycle time, throughput) on a cadence.
Changing requirements	Good between sprints; changes mid-sprint are restricted.	Very good; reprioritise any time.
Cross-functional work	Strong – whole team owns a sprint goal.	Good – but no shared goal unless added.
Limitations / overhead	Ceremony overhead; no velocity history makes early forecasts weak; stakeholders (2 h/week) may be over-stretched.	No timebox, so stakeholders get no predictable review points; harder to commit to a six-week release without a cadence.
Delivery risks	Over-commitment in Sprint 1; mid-sprint urgent work.	Drift and weak release forecasting; hidden dependencies.

2.2 Selected approach and tailored operating model

Selection: “Thin-Slice Flow Model” – Scrum cadence + Lean flow controls + selected XP practices. Scrum gives the release a fixed rhythm (three sprints, sprint goals, stakeholder reviews that fit the 2 h/week limit). Lean controls (WIP limits, explicit policies, blocked flags) attack the actual risks here: waiting for clarifications/access, handovers and rework. XP practices (pairing on RBAC and SLA logic, automated tests, continuous integration) reduce defects in the logic that the assignment says must be verified.

Element	Operating model definition
Roles & responsibilities	Product Owner/BA: owns backlog order, acceptance, decision log (SLA, roles). Scrum Master/delivery lead: facilitation, impediments, flow metrics, escalation. Backend (3): APIs, SLA/priority logic, audit, integration adapter. Frontend (1): intake, queue, approval screens. DevOps (1): CI/CD, environments, mock tracker, access. QA (1): test design, role matrix, regression; developers also test (“dev-in-test”) to avoid a QA bottleneck.
Planning activities	Release planning (week 0), sprint planning (2 h), weekly backlog refinement (30 min), capacity check at start of each sprint.

Element	Operating model definition
Review points	Mid-sprint review (45 min, Thursday of week 1), sprint review/demo (1 h), retrospective (45 min), daily 15-min stand-up focusing on blocked and ageing items.
Stakeholder participation (2 h/week budget)	Week budget: 30 min clarification office hour (analyst + PO), 30 min CAB/manager policy decisions, 1 h review/demo (alternating weeks). Decisions are recorded in a Jira decision log so they are never asked twice.
Escalation procedure	Level 1: team swarms within 4 hours. Level 2: Scrum Master/PO resolve within 1 working day. Level 3: sponsor/steering decides within 2 working days. Any item blocked > 1 day is flagged red on the board.

2.3 How Lean principles reduce waste

Waste	Lean practice in this project	Evidence of effect
Waiting	Definition of Ready requires decisions (SLA/role matrix) before a story starts; pre-booked clarification slots; mock issue tracker avoids waiting for week-3 access.	Blocked time per sprint
Handovers	Vertical slices owned by a pair (backend + frontend); developer writes and runs tests; no separate "throw-over" to QA.	Number of status transitions; review wait time
Rework	Acceptance criteria written before work; automated tests on priority/SLA logic; early demo of intake form.	Reopened-story rate
Excess WIP	WIP limits per column (Task 4); "stop starting, start finishing".	Average WIP, cycle time

2.4 Project-management tool investigation

Criterion	Jira	Trello	GitHub Projects	Azure DevOps
Planning (epics, stories, sprints)	Native epics, sprints, backlog, roadmap	Cards/lists only; sprints need add-ons	Iterations and fields; light epics	Strong: epics, features, sprints
Workflow management	Custom statuses, transitions, conditions, WIP limits (visual)	Simple lists; limited rules	Board columns; basic automation	Custom workflows and rules
Collaboration	Comments, links, mentions, Confluence, Slack integration	Easy comments and attachments	Tied to repos, PRs, issues	Strong with repos/pipelines
Reporting	Burndown, velocity, cumulative flow, control chart	Basic, via Power-Ups	Insights charts, limited agile metrics	Velocity, CFD, dashboards

Criterion	Jira	Trello	GitHub Projects	Azure DevOps
Accessibility	Keyboard navigation and published accessibility documentation (verify current status)	Reasonable; simpler UI	Good keyboard support (GitHub)	Good; Microsoft accessibility practices
Cost	Free up to 10 users; paid per user	Free tier; paid for advanced	Free for public/small teams; paid org plans	Free for 5 users; paid beyond

Decision: Jira. It matches the assignment (CO1 names Jira) and provides the three things the strategy needs: custom workflow with explicit transitions, issue links for dependencies, and flow reports (cumulative flow, control chart, velocity) that give the evidence used in Tasks 5 and 6. Its team-managed projects and "Team"/component fields also support two teams in Task 6.

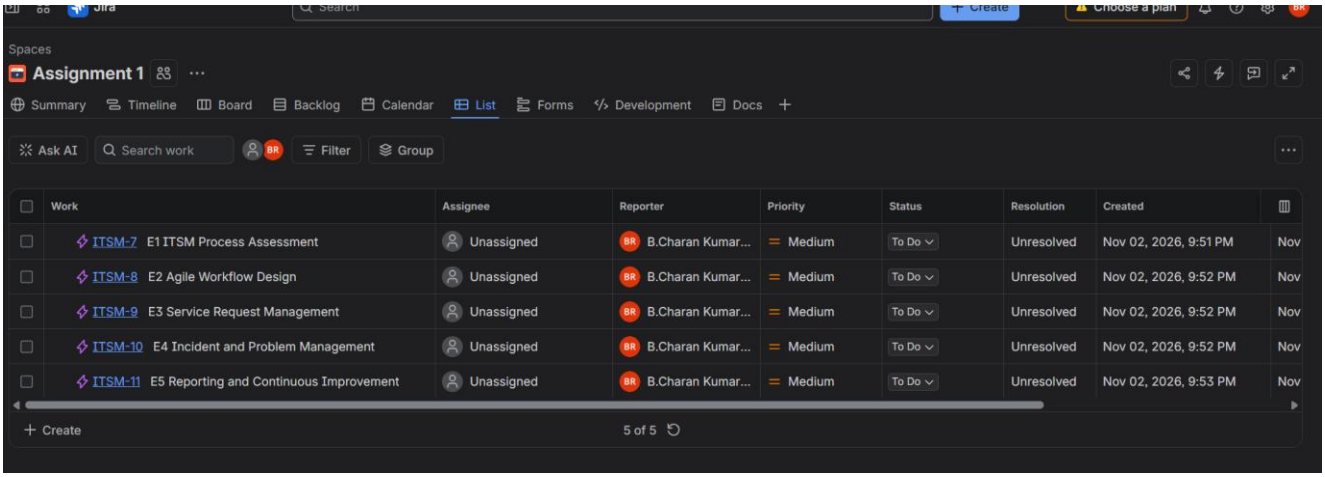
Limitations and how I address them: (1) WIP limits in Jira are visual warnings, not hard blocks → working agreement plus an automation rule that comments/flags when a limit is exceeded. (2) Over-customisation slows set-up → use one workflow, five custom fields (Story Points, Role, Dependency, Team, Decision Needed). (3) Cost grows beyond 10 users → budget the Standard plan before scaling. (4) Learning curve → 30-minute onboarding and a one-page board guide.

Task 3 – Prioritized Product Backlog in Jira

A Jira Scrum project "ITSM" (key ITSM) is created with five epics and fourteen first-release stories. All statuses, estimates and links below are exactly what is entered in Jira; screenshots/export are listed in Appendix A.

3.1 Epics and module decisions

Jira Evidence (Task 3): Epic and work-item list in the ITSM project.



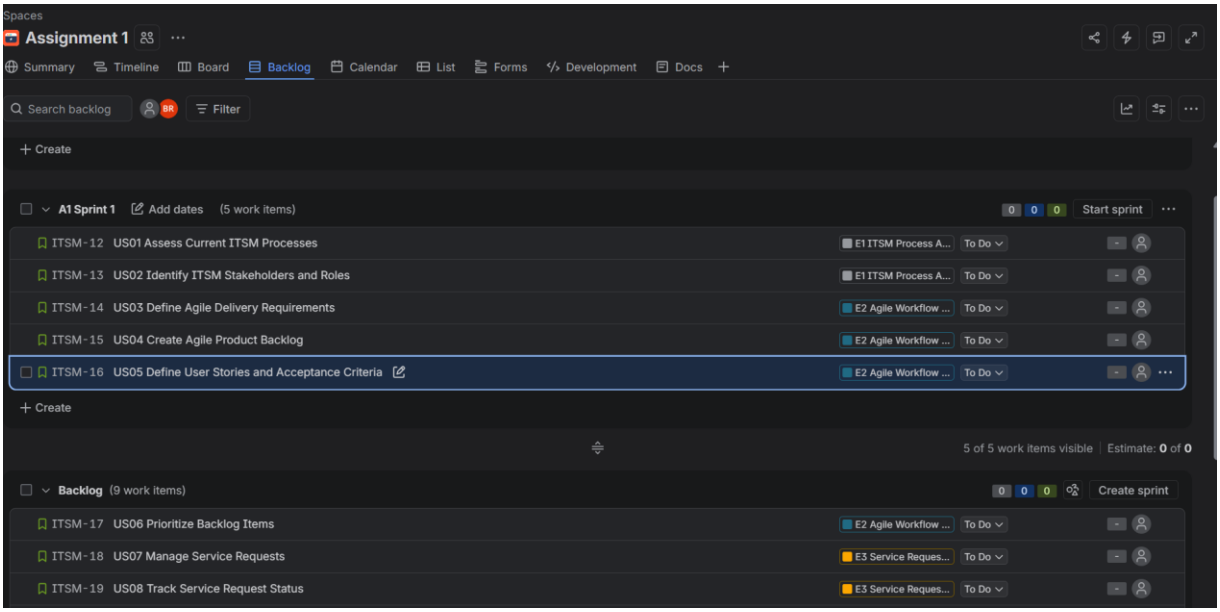
Epic	Name	Purpose	Modules covered
ITSM-E1	Incident Management & SLA	Intake, priority, SLA, assignment, lifecycle, analyst queue	Incident lifecycle, reduced dashboard
ITSM-E2	Triage & Telemetry	Telemetry events linked to incidents; rule-based suggestions	Telemetry, assisted triage
ITSM-E3	Approval Governance	Change and manager approvals with audit	Change approval (service-request approval only)
ITSM-E4	DevOps Integration & Closure	Developer issue link, safe retry, deployment verification	DevOps integration hub
ITSM-E5	Platform Foundations	Role-based access, audit log, privacy/accessibility	Cross-cutting

Module (8)	Decision	Rationale
Operations dashboard	Retained – reduced	Analyst queue + SLA indicator only; analytics deferred.
Assisted triage & diagnostics	Retained – thin (Should)	Keyword rules only; labelled deterministic; first to slip if capacity is tight.
Incident lifecycle management	Retained – full	Core of the journey.
Infrastructure telemetry	Retained – thin	Synthetic events + one correlation rule.
Service request catalog	Deferred	Not on the critical incident path; only manager approval story kept.
Change approval	Retained	Managers/CAB need auditable approvals.

Epic	Name	Purpose	Modules covered
Problem investigation	Deferred	Linked later; incident has a “problem reference” text field only.	
DevOps integration hub	Retained – thin	Issue link + local persistence + retry; no deep sync.	

3.2 Prioritized backlog (estimates in story points, relative size – not hours)

Jira Evidence (Task 3): Backlog with work items allocated to sprints. Story-point estimates are not visible here.



ID	Epic	User story	MoSCoW	Pts	Depends on	Role	Sprint
US-01	E5	As an administrator, I want six roles with permissions so that users only perform allowed actions.	Must	8	–	BE, QA	S1
US-02	E5	As a manager, I want every state change audit-logged so that decisions can be traced.	Must	5	US-01	BE	S1
US-03	E1	As an employee, I want to report an incident through a form so that it gets a tracked ID.	Must	5	US-01	BE, FE	S1
US-04	E1	As an analyst, I want priority and SLA deadlines calculated automatically so that I can act on urgent items.	Must	5	US-03	BE	S1
US-05	E1	As an analyst, I want to assign and move incidents through a lifecycle so that ownership is clear.	Must	5	US-03, US-04	BE, FE	S2

ID	Epic	User story	MoSCoW	Pts	Depends on	Role	Sprint
US-06	E1	As an analyst, I want a queue showing SLA status so that I can see breaches coming.	Must	5	US-04, US-05	FE	S2
US-07	E2	As an SRE, I want synthetic telemetry events ingested so that I can see signals.	Must	5	–	BE, DevOps	S2
US-08	E2	As an SRE, I want telemetry linked to incidents by a rule so that correlation is not manual.	Must	8	US-03, US-07	BE	S2
US-09	E2	As an analyst, I want keyword-based diagnostic suggestions so that triage is faster.	Should	3	US-08	BE	S3
US-10	E3	As an engineer, I want to raise a change request linked to an incident so that changes are justified.	Must	5	US-05	BE, FE	S3
US-11	E3	As a CAB reviewer, I want to approve or reject a change with a reason so that approvals are auditable.	Must	5	US-10, US-01, US-02	BE, FE	S3
US-12	E3	As a manager, I want to approve service requests so that access is controlled.	Should	3	US-01, US-02	BE	S3
US-13	E4	As an SRE, I want to link an incident to a developer issue, with local save and safe retry if the tracker is down.	Must	8	US-05; tracker access (wk 3)	BE, DevOps	S2
US-14	E4	As a manager, I want deployment verification recorded before closure so that fixes are confirmed.	Must	3	US-11, US-13	BE, QA	S3

Total 73 points. Deferred (Could, Release 2): service request catalog, problem investigation module, dashboard analytics, ML-based diagnostics (PROPOSED).

3.3 Acceptance criteria (Given / When / Then, abbreviated)

Story	Acceptance criteria
US-01	Given a user with role X, when they request an action outside X, then the system denies it and logs the attempt. The 40-case role matrix passes with 0 unauthorised actions.
US-02	Every create/update/approve/close action stores user, time, old and new value; audit records cannot be edited via the UI.
US-03	Given a completed form, when submitted, then a unique ID is shown and the record persists after restart; empty description/category is rejected with an accessible error message.
US-04	Priority follows the impact × urgency matrix; SLA due time matches the P1–P4 table in 100% of 20 test cases; paused/resumed states handled.

Story	Acceptance criteria
US-05	Only allowed transitions (New→Triaged→Assigned→Investigating→Change Pending→Resolved→Closed) are possible; an owner is always shown.
US-06	Queue lists 500 synthetic incidents sorted by SLA remaining; breached items highlighted using text + colour (not colour only); keyboard operable.
US-07	Controlled sample of 2,000 events loads; malformed events are rejected and counted, not crashing the import.
US-08	A defined rule (same service + time window) attaches an event to an incident; 10 of 10 test cases link correctly; the rule is documented as deterministic.
US-09	Suggestions come from a keyword table; each suggestion is labelled “rule-based”; analyst must confirm before use.
US-10	A change request cannot be created without a linked incident and justification; status is “Pending CAB”.
US-11	Only CAB reviewer role can approve/reject; reason is mandatory; requester cannot approve own change; audit entry created.
US-12	Manager can approve/reject a service request; non-managers cannot; decision is audited.
US-13	When the mock tracker is unavailable, the link request is saved locally with “Pending sync”; retries run with back-off and do not create duplicates (idempotent key).
US-14	Incident cannot move to Closed without a verification note and an approved change (if one exists).

3.4 Prioritization method

I used MoSCoW to decide what is in the release and WSJF-lite (Business value + Time criticality/Risk reduction, divided by job size; each scored 1–10 relative scores, ASSUMPTION) to order stories. Dependencies then override the raw score: foundations (US-01) must come first even though their score is lower.

Story	Value	Time/Risk	Size	WSJF	Story	Value	Time/Risk	Size	WSJF
US-01	8	10	8	2.3	US-08	9	6	8	1.9
US-02	7	8	5	3.0	US-09	5	3	3	2.7
US-03	10	9	5	3.8	US-10	7	5	5	2.4
US-04	9	8	5	3.4	US-11	8	6	5	2.8
US-05	8	7	5	3.0	US-12	5	3	3	2.7
US-06	9	6	5	3.0	US-13	8	8	8	2.0
US-07	7	7	5	2.8	US-14	6	4	3	3.3

3.5 Definition of Ready and Definition of Done

Definition of Ready (a story may enter a sprint only if)

- Story has a role, goal, value and Given/When/Then acceptance criteria.
- Required decisions (e.g. SLA matrix, role permissions) are recorded in the decision log.
- Dependencies are identified and either done or have a stub/mock agreed.
- Estimated by the team (≤ 8 points; larger stories are split); test data identified.
- Privacy and accessibility notes present (what data is used, which screens).

Definition of Done (a story is Done only if)

- Acceptance criteria demonstrated; code peer-reviewed and merged; CI build green.
- Automated tests pass; ≥ 70% coverage on SLA/priority/permission logic (ASSUMPTION threshold); no open Sev-1/2 defects.
- Privacy: only synthetic data; no personal data in logs or screenshots; access follows role matrix.
- Accessibility: keyboard operable, labelled form fields, contrast ≥ 4.5:1, status not conveyed by colour alone (WCAG 2.1 AA-oriented check).
- Audit entries verified for state changes; documentation/Jira updated; demonstrated to PO.

3.6 Decisions where competing stakeholder requests shaped priorities

1. Analysts vs SREs. Analysts wanted intake/SLA visibility; SREs wanted telemetry correlation. I scheduled intake, priority and SLA in Sprint 1 (everything depends on a tracked incident) and gave SREs raw telemetry ingestion plus one correlation rule in Sprint 2. Advanced correlation was deferred. SREs get value mid-release, analysts get value first.
2. Management (all eight modules) vs managers (approvals). Rather than building eight shallow modules, I kept approvals (US-10/11/12) and deferred the service-request catalog and problem module, because auditable approval is a named need and part of the incident journey, while the catalog is not.
3. Developers vs delivery risk. Developers wanted full tracker synchronisation. I kept only link + local persistence + safe retry, because access starts in week 3 and local records must survive tracker outage.

Task 4 – Capacity-Based Release Plan and Delivery Workflow

4.1 Capacity assumptions (ASSUMPTION)

Item	Sprint 1	Sprint 2	Sprint 3
Delivery-capable people (3 BE, 1 FE, 1 DevOps, 1 QA)	6	6	6
Working days	10	10	10
Focus factor (meetings, reviews, support)	0.6 (new team, setup)	0.7	0.7 (+ hardening time)
Productive person-days	36	42	42
Calibration (person-days per point, starting guess)	1.5	1.35	1.35
Forecast capacity (points)	≈ 24	≈ 31	≈ 31 (25 usable after hardening)
Committed	23	31	19 + buffer

PO and Scrum Master are excluded from delivery capacity. Capacity reductions for leave are checked at each planning session.

4.2 Estimation method

The team uses relative estimation (planning poker, Fibonacci scale 1-2-3-5-8) against reference stories: US-03 (intake form, 5) is the “medium” anchor. Points express effort, complexity and uncertainty together; they are not fixed hours. The “person-days per point” figures above are only a starting calibration to avoid over-committing in Sprint 1 with no velocity history; they are replaced by observed velocity after Sprint 1. Forecasts are given as ranges. Stories above 8 are split.

4.3 Sprint goals and plan

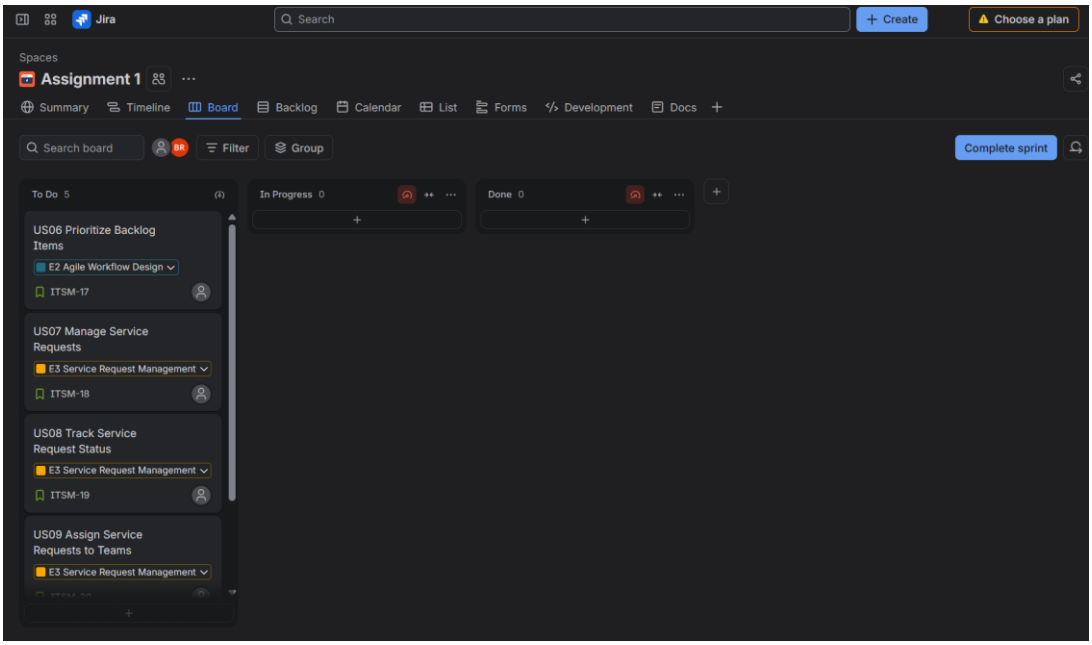
Sprint	Goal	Stories (points)	Dependency notes
1 (weeks 1–2)	A user can report an incident and get a tracked ID with correct priority and SLA, under enforced roles and audit.	US-01 (8), US-02 (5), US-03 (5), US-04 (5) = 23	US-01 stub early so US-02/US-03 can start.
2 (weeks 3–4)	An analyst can own and move incidents with SLA visibility, and an SRE can link telemetry and a developer issue.	US-05 (5), US-06 (5), US-07 (5), US-08 (8), US-13 (8) = 31	US-13 uses real tracker access starting week 3; mock until then.
3 (weeks 5–6)	A change can be justified, approved and verified so an incident can be closed end to end.	US-10 (5), US-11 (5), US-12 (3), US-14 (3), US-09 (3) = 19 + hardening/E2E tests	US-09 first to drop if capacity is lost.

4.4 Dependency sequencing and responsibilities

Critical path	Cross-functional responsibility
US-01 RBAC → US-03 intake → US-04 SLA → US-05 lifecycle → US-10 change → US-11 CAB approval → US-14 verification/closure Parallel path: US-07 telemetry → US-08 correlation → US-09 suggestions External path: tracker access (week 3) → US-13 link/retry → US-14	Backend: logic and APIs; Frontend: screens and accessibility; DevOps: environments, mock tracker, CI; QA: role matrix, regression; PO: decisions and acceptance; SM: flow and impediments.

4.5 Development workflow in Jira (distinct from the incident lifecycle)

Jira Evidence (Task 4): Board with To Do, In Progress and Done columns.



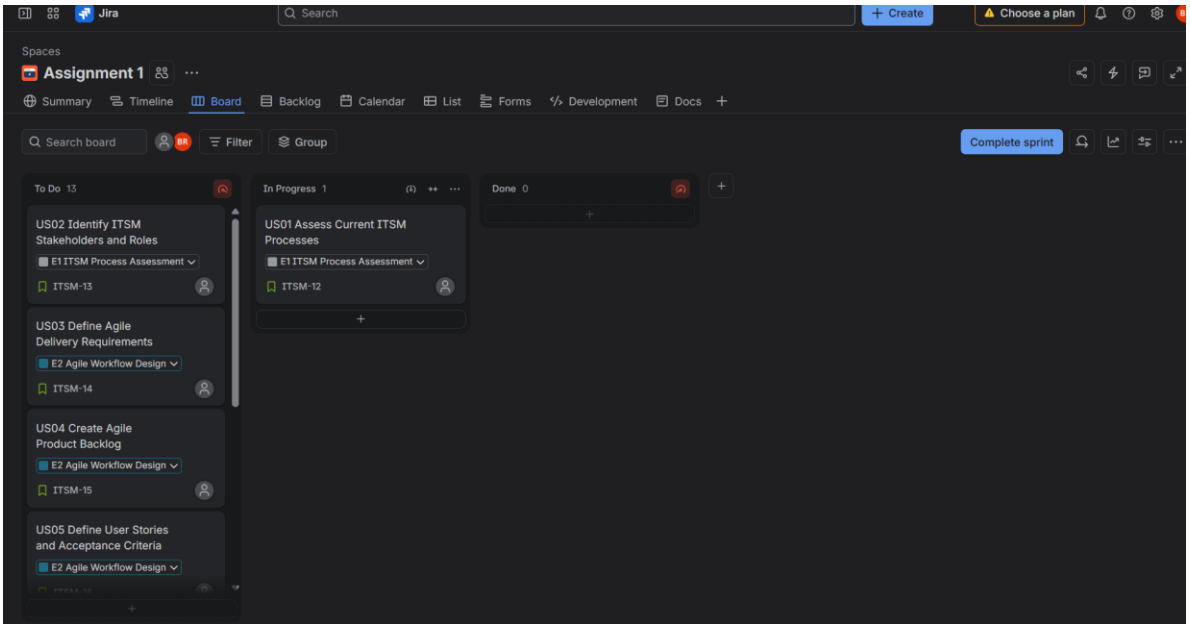
The development workflow tracks how a Jira story is built. The operational incident lifecycle (New → Triaged → Assigned → Investigating → Change Pending → Resolved → Closed) is a feature of the platform being built (US-05); the two must not be confused.

Status	Entry rule (transition condition)	WIP limit	Justification
Backlog	Any story in the product backlog	–	Unlimited, ordered by PO.
Ready	Definition of Ready checklist complete; dependencies resolved/stubbed	6	About one half sprint of work; avoids stale refined stories.
In Progress	Owner assigned; pair identified; sprint-committed	5	Six delivery people but pairing means ≤ 5 items can be actively worked; forces finishing.
In Review	Code complete; PR open; tests written	3	Reviews should happen within a day; limit forces swarming on reviews.

Status	Entry rule (transition condition)	WIP limit	Justification
In Testing	Reviewed and merged to test branch	2	One QA plus dev-in-test partner; prevents a testing queue.
Done	Definition of Done checklist ticked; PO accepts	–	Reopen allowed only with a defect link.
Blocked (flag)	Impediment text and owner required; red flag	Escalates after 1 day	Makes waiting visible (Lean "andon").

Handling of blocked, urgent and unfinished work

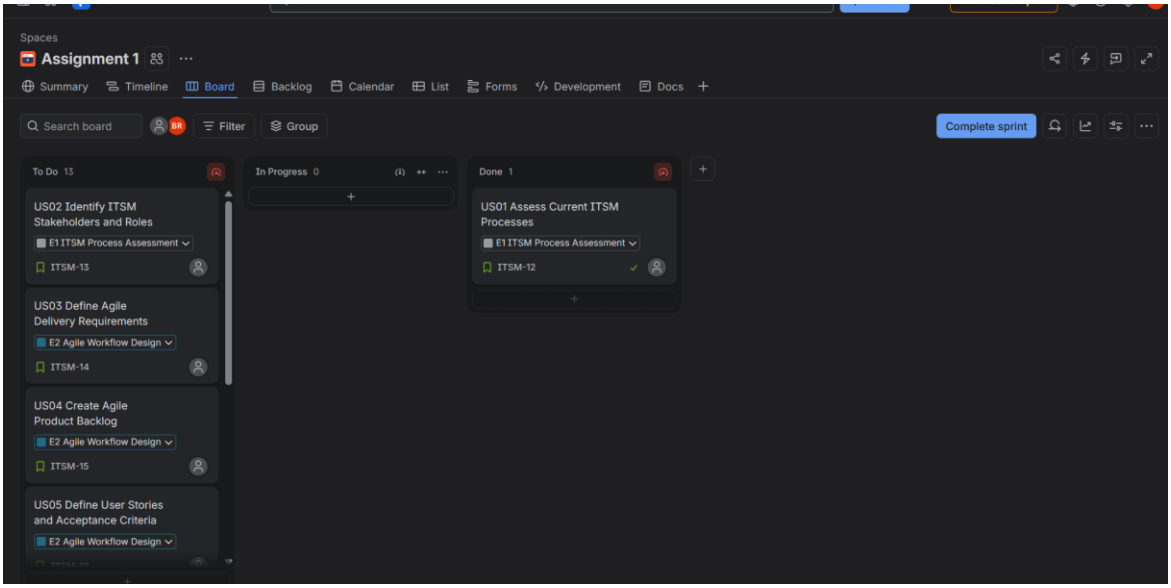
Jira Evidence (Task 4): Sprint board work-item status view.



- Blocked: flag with reason and owner; swarm for 4 h (Level 1); if still blocked after 1 day escalate to Level 2; blocked days are recorded for the indicator.
- Urgent requests: an "expedite" lane allows at most one urgent item at a time; PO decides which committed story leaves the sprint; 10% of capacity is held as buffer in Sprint 2 and 3.
- Unfinished stories: not silently carried; they are re-estimated for remaining work, returned to the top of the backlog, counted as incomplete for velocity, and discussed in the retrospective.

4.6 How the selected features achieve the product goal

Jira Evidence (Task 4): Board status-column view.



Journey step	Feature (story)	Sprint
Report	Intake form (US-03) under roles (US-01)	1
Track and prioritise	Priority + SLA (US-04), audit (US-02)	1
Own and investigate	Lifecycle + queue (US-05, US-06), telemetry link (US-07/08)	2
Justify and approve change	Change request + CAB approval (US-10/11)	3
Link development work	Dev-issue link with retry (US-13)	2
Verify and close	Deployment verification (US-14)	3

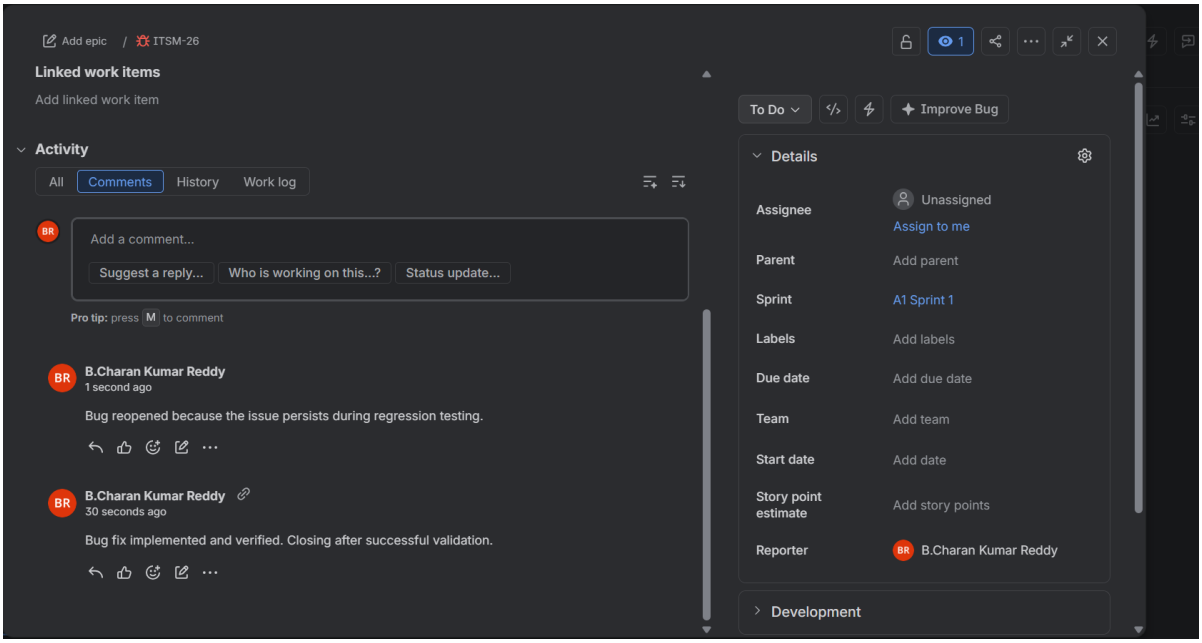
Task 5 – Simulate Delivery and Validate the Design

5.1 Prediction made before simulation

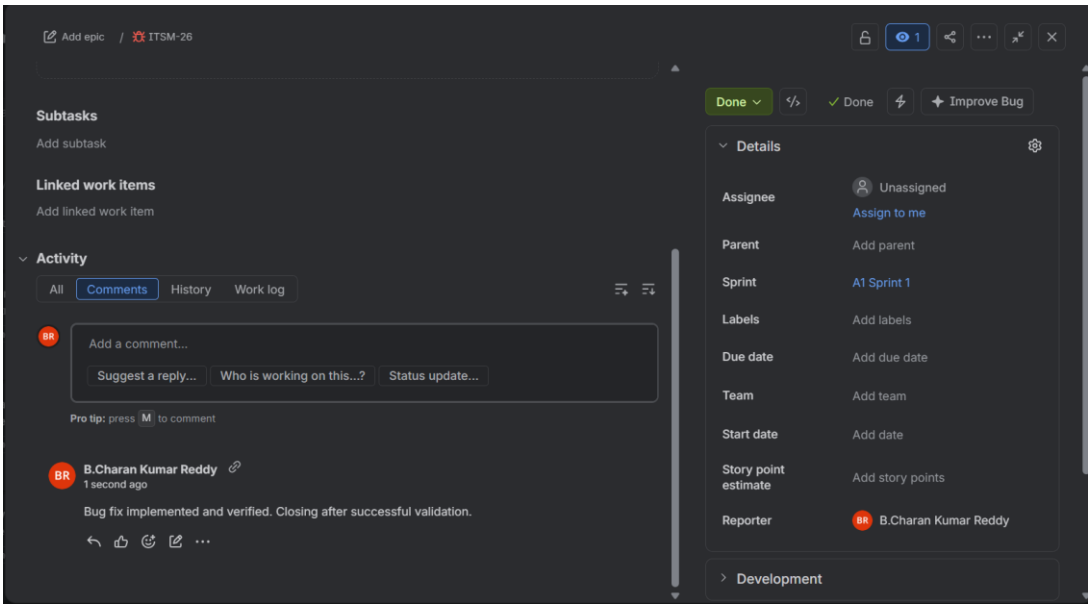
Item	Detail
Predicted bottleneck	In Review / In Testing: with a single QA and several stories finishing together, work will queue before testing.
Controlling rule	WIP limits of 3 (In Review) and 2 (In Testing), swarming on reviews and dev-in-test.
Evidence the rule works	Average wait in review ≤ 1 day; WIP at or below limits; no column exceeds its limit for more than half a day.

5.2 Event log (Sprint 1, SIMULATED)

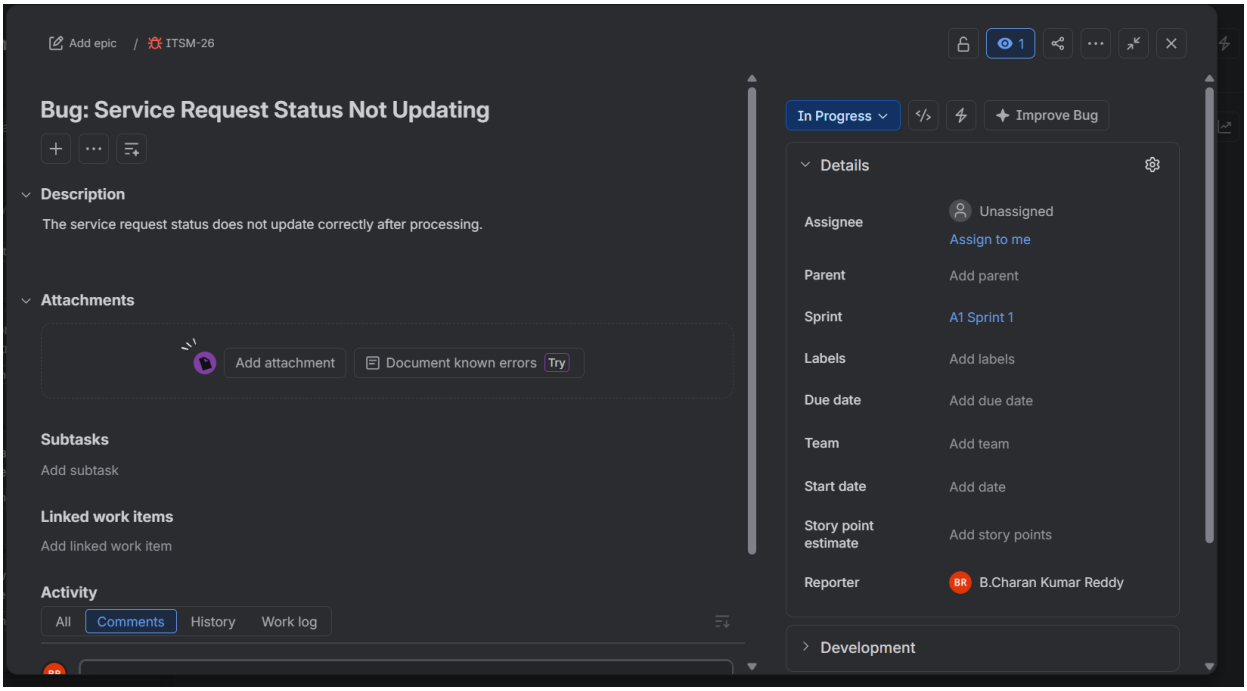
Jira Evidence (Task 5): Bug reopened after regression testing.



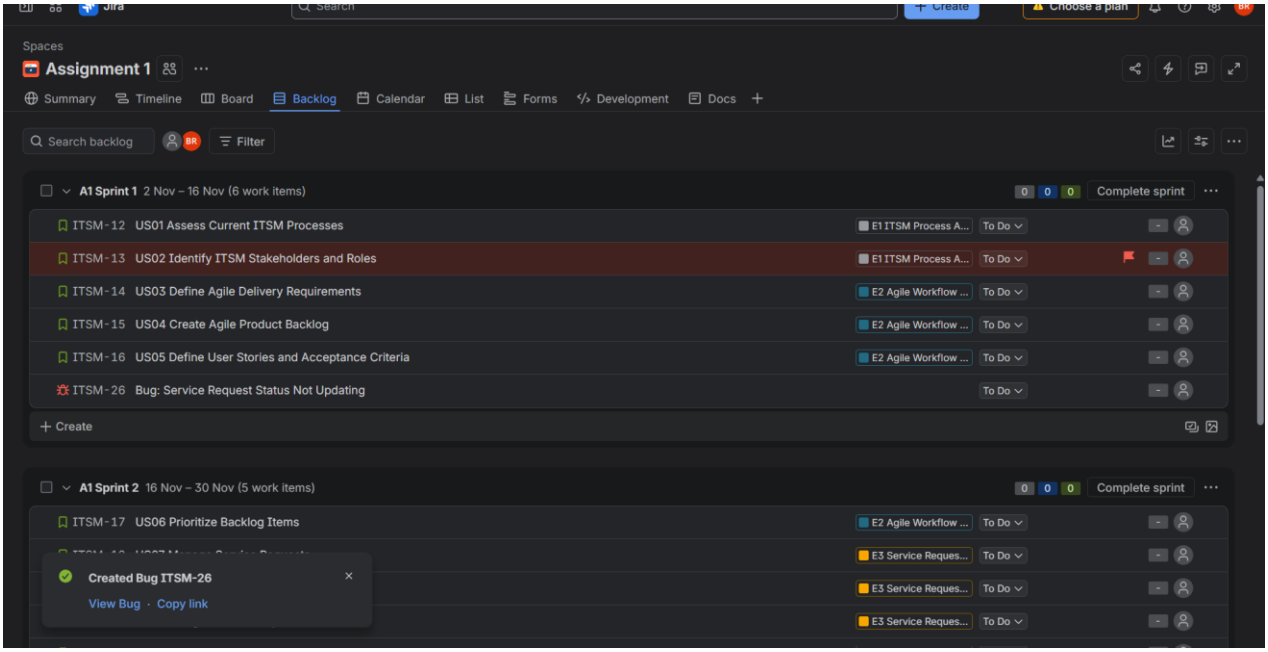
Jira Evidence (Task 5): Bug marked Done after simulated verification.



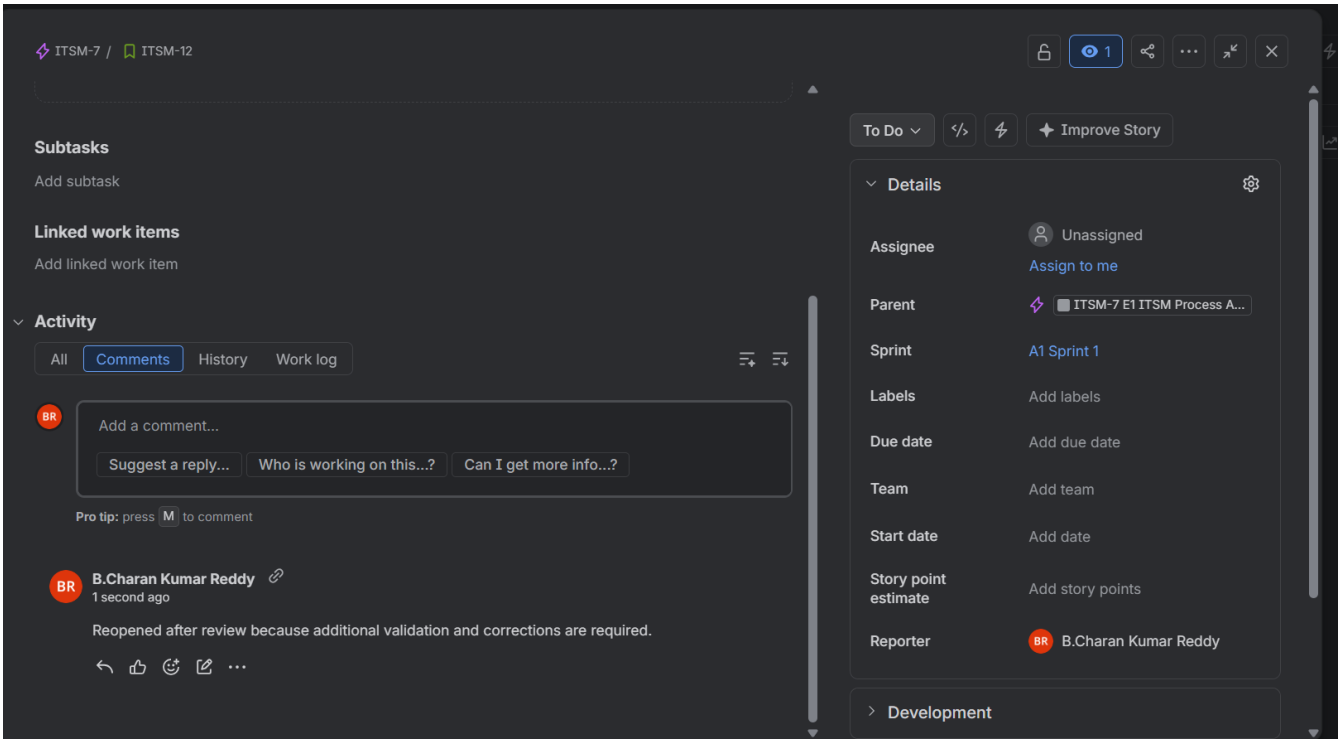
Jira Evidence (Task 5): Bug ITSM-26 – Service Request Status Not Updating.



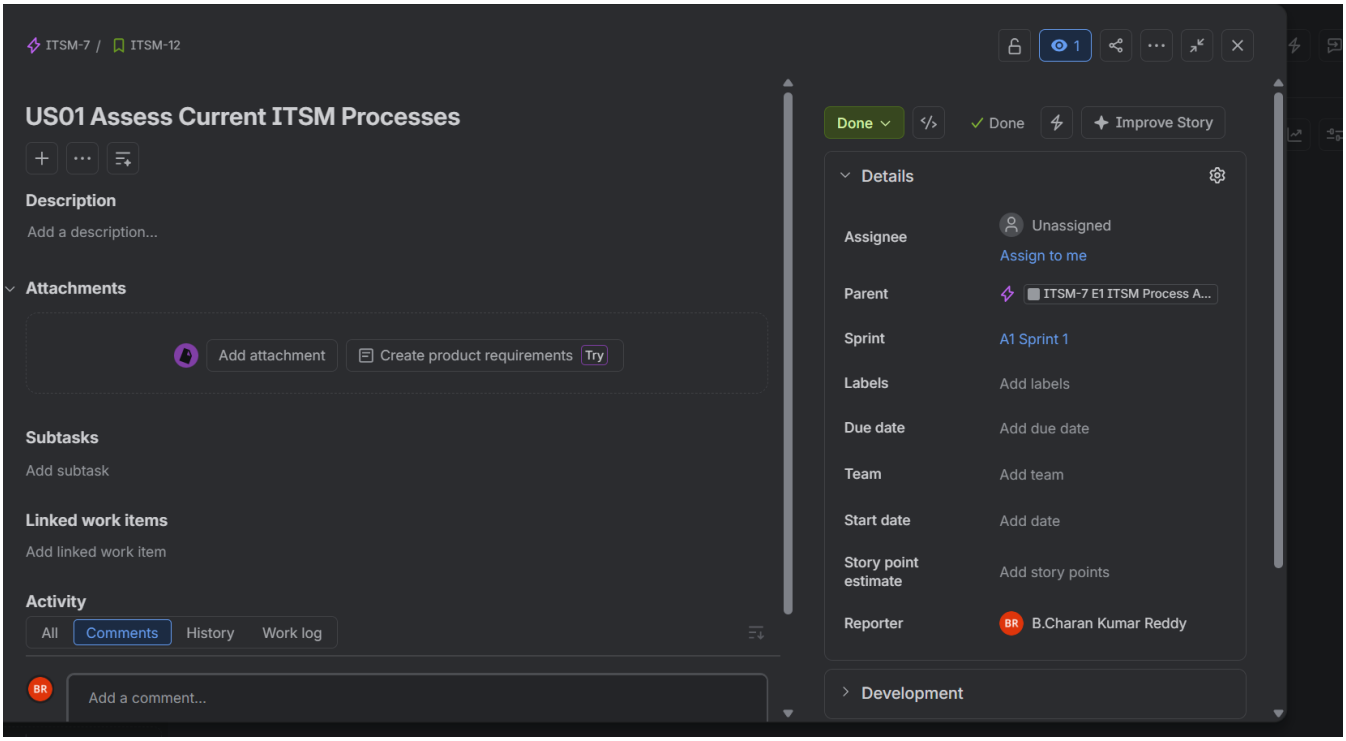
Jira Evidence (Task 5): Sprint backlog listing.



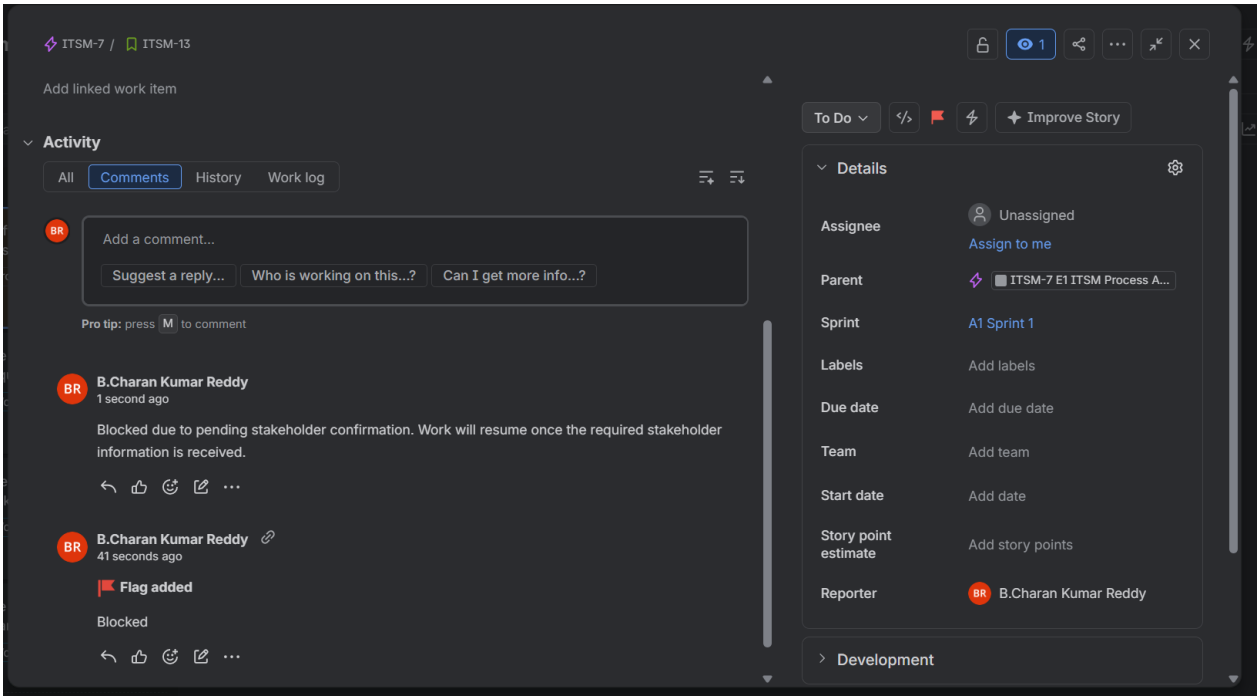
Jira Evidence (Task 5): Reopened story after review.



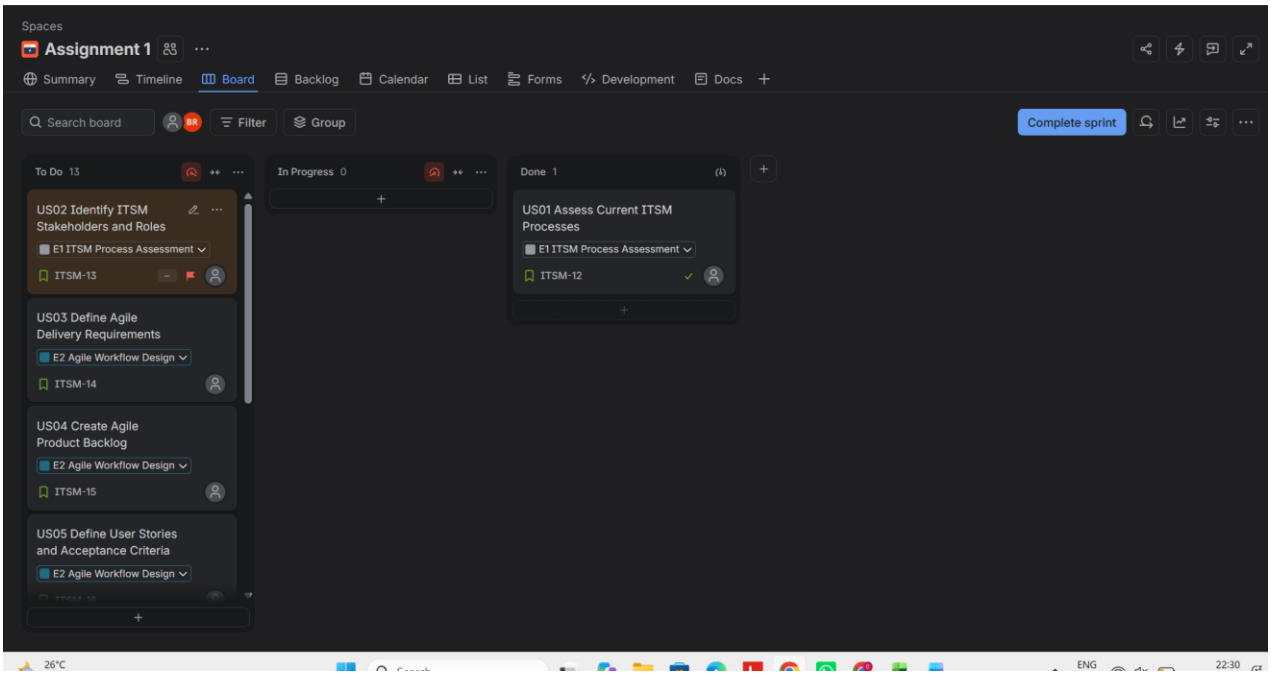
Jira Evidence (Task 5): US01 issue status view.



Jira Evidence (Task 5): Blocker comment on US02.



Jira Evidence (Task 5): Sprint board with flagged/blocked item.



#	Day	Event	Story	Jira update / effect
1	D1	Sprint planning: commit 23 points; WIP limits loaded	All	Sprint started; 4 stories moved to Ready
2	D1	Work starts on role-permission model	US-01	To In Progress
3	D2	Dependency identified: US-02, US-03 need RBAC; decision: BE1 publishes a permission-interface stub	US-01/02/03	Issue links “is blocked by” set; stub due D3
4	D3	Intake form work begins using the stub	US-03	To In Progress
5	D3	SLA matrix ambiguity (“partial outage” priority) found; analyst slot not until D5	US-04	Flagged Blocked (clarification)
6	D4	Audit log started; CAB reviewer test accounts missing	US-02, US-01	US-01 flagged Blocked 1 day; DevOps seeds accounts
7	D5	Clarification received; decision logged (partial outage = P2)	US-04	Unblocked after 2 days
8	D5	Defect: empty description accepted by intake form	US-03	Reopened; bug linked
9	D6	In Review reaches limit (3); team stops starting and swarms on reviews	US-01/03/04	WIP rule triggered
10	D6	Defect fixed and retested	US-03	Done (cycle time 4 days)
11	D8	RBAC verified with 40-case role matrix	US-01	Done (cycle time 8 days)
12	D9	Audit log accepted by PO	US-02	Done (cycle time 6 days)
13	D10	SLA timer UI awaiting QA; sprint review and retrospective held	US-04	Incomplete; returned to backlog (5 pts)

5.3 Delivery indicators (SIMULATED)

Indicator	Value	Comment
Completed vs planned	18 / 23 points (78%) – 3 of 4 stories	Just below the 80% target (SC1).
Blocked time	3 days (US-04: 2, US-01: 1)	Within ≤ 4 days (SC3).
Cycle time (avg of Done stories)	6.0 days (4, 6, 8)	Above the 5-day median target.
Average WIP	4.2 (limit 5)	Limit respected; review column hit its limit once.
Reopened stories	1 of 3 (33%)	Missing validation case; add to DoD.
Average wait in review	1.5 days	Above ≤ 1 day target.

5.4 Interpretation, limitation and improvement

Interpretation. The prediction was partly correct: the review column reached its WIP limit and reviews waited 1.5 days. But the larger delay came from elsewhere: a two-day wait for an SLA clarification and the 8-point RBAC story that sat on the critical path. The WIP rule exposed the review queue but did not remove waiting for decisions.

Limitation in the original plan. I assumed that 2 hours per week of stakeholder time was enough, but did not schedule decisions in advance, and RBAC was too large to be a single prerequisite.

- Improvement 1: the Definition of Ready now requires an entry in the decision log; a decision workshop for the Sprint 2 and 3 rules (change approval matrix, retry policy) is held in the final week of the previous sprint.
- Improvement 2: stories that act as prerequisites are split so a stub is available by day 2.
- Improvement 3: a second tester is trained (dev-in-test pairing) and the DoD adds a “negative/empty input” test check to cut reopened stories.
- Sprint 2 plan: US-04 (5 carried) joins the sprint; the forecast uses the observed 18-point velocity as the lower bound and the 24-point estimate as the upper bound.

Task 6 – Redesign Delivery Under Changed Conditions

A. Diagnose the impact

Aspect	Original assumption	After change	Impact / risk
Teams and capacity	One team, 6 delivery people	Team A: 5 delivery people (one backend lost); Team B: 6 new people	A: ≈ 26 pts/sprint (−5). B: ramp-up at 0.5 productivity in Sprint 2, 0.8 in Sprint 3 (ASSUMPTION): ≈ 15 then ≈ 25 pts.
Scope	73 pts total; 55 pts remaining after Sprint 1 (5 carried + 31 + 19)	+34 pts of new work (BU SLA policies 8, BU approvals 8, BU data/role scoping 8, onboarding 5, cross-team integration tests 5)	Demand 89 pts vs net capacity ≈ 78 pts.
Dependencies	Internal + tracker access	Shared code and database between teams; BU policy decisions	Merge conflicts, cross-team waiting; more communication paths (28 → 120).
Stakeholder time	2 h/week	Still 2 h/week but 4 BU policy owners	Policy decisions are a bigger bottleneck (Sprint 1 showed 2-day waits).
Deadline	Week 6	Unchanged	Must cut or simplify scope, not extend time.

Net capacity (Sprint 2): (26 + 15) × 0.85 coordination factor ≈ 35. Sprint 3: (26 + 25) × 0.85 ≈ 43. Total ≈ 78 points (ASSUMPTION). Top risks (ROAM): onboarding lag (Mitigate), BU policy conflicts (Resolve through workshops), integration conflicts between teams (Mitigate through contract-first APIs), loss of the backend developer (Accept and re-plan).

B. Redesign objectives

ID	Measurable objective	Target
R1	Scope: committed work vs net capacity; Must stories retained	≤ 90% of net capacity (≤ 70 pts); 100% of Must kept
R2	Predictability per team per sprint	≥ 85% of planned points completed
R3	Coordination: average wait on cross-team dependency; daily integration build success	≤ 1 day; ≥ 90% green
R4	Journey: end-to-end scenarios across 3 BU profiles	12 of 12 pass; BU SLA and approvals configured for all three BUs
R5	Quality: reopened-story rate; blocked days per team per sprint	≤ 15%; ≤ 4 days

C. Revised strategy

C1. Scope reduction to fit capacity (WSJF re-scored)

Change	Points saved	Reason
Defer US-09 keyword suggestions to Release 2	3	Should, lowest WSJF in the remaining set.
Merge US-12 into BU approval story N2	3	Overlapping work.
Reduce US-08 to a single correlation rule (8 → 5)	3	Advanced rules not needed for the journey.
US-14 simplified to a verification note + checklist (3 → 1)	2	Still preserves closure check.
N1 BU SLA policies delivered as one configuration table with 3 BU profiles (8 → 5)	3	Configuration instead of custom code.
N2 BU approvals via template approval chain (8 → 5)	3	Template approach.
N4 onboarding as a short guide (5 → 3), N5 tests limited to critical path (5 → 3)	4	Lean documentation and test focus.

Result: 89 − 21 = 68 points, which is 87% of the 78-point net capacity, leaving a 13% buffer.

C2. Team structure and responsibilities

Team	Stream and ownership	Stories
Team A (7 members: PO/Scrum Master roles, 2 BE, 1 FE, 1 DevOps, 1 QA – one BE lost)	Incident & Governance stream: incident lifecycle, SLA, approvals, BU policy/approval configuration, data scoping	US-04, US-05, US-06, N3a (S2); US-11, N1, N2, N3b (S3)
Team B (8 members, new)	Telemetry & Integration stream: telemetry, correlation, developer link, change request, verification, onboarding and cross-team tests	US-07, US-13 (S2); US-08, US-10, US-14, N4, N5 (S3)

New items: N1 BU-aware SLA configuration (5), N2 BU service-request approvals (5), N3a BU role scoping (5, S2), N3b data-separation tests (3, S3), N4 BU onboarding guide (3), N5 cross-team integration tests (3).

C3. Revised sprint sequencing

Sprint	Team A (points)	Team B (points)	Total / capacity
2	US-04 5, US-05 5, US-06 5, N3a 5 = 20	US-07 5, US-13 8 = 13	33 / ≈ 35 (94%)
3	US-11 5, N1 5, N2 5, N3b 3 = 18	US-08 5, US-10 5, US-14 1, N4 3, N5 3 = 17	35 / ≈ 43 (81%)

Sprint 2 is the riskiest (94% load). If it slips, US-06 moves to Sprint 3, which has 19% headroom.

C4. Scaled Agile and Lean practices adopted (SAFe-inspired, "lite")

Practice	What I adopt	Why it fits
Joint planning (PI-planning-lite)	A 2-hour combined planning for both teams at the start of Sprint 2; shared sprint goals and a dependency board.	Two teams, one release; prevents conflicting plans.
Scrum of Scrums	15 min, 3 times a week, one representative per team; flags cross-team blockers.	Cheap coordination at 16 people.
Shared backlog with "Team" field	One Jira project, Team field and component per stream; PO prioritises across both.	Single source of truth.
Contract-first APIs and daily integration build	Interfaces agreed before coding; one integration environment.	Avoids late merge surprises.
System demo and Inspect & Adapt	Joint demo at sprint end; joint retrospective with metrics.	Evidence-based improvement.
Lean controls	WIP limits per team, ROAM board for risks, expedite lane, decision log, buddy pairing to onboard Team B.	Continues the Task 2 Lean approach.

Full SAFe roles and Program Increment ceremonies are not adopted because they would add overhead disproportionate to a six-week release of two teams.

D. Validate the redesign

#	Day	Simulated event (revised Sprint 2)	Effect
1	D1	Joint planning of both teams; 33 pts committed; Team field set in Jira	Dependency board created
2	D2	Team B setup issue (environment access) delays start by 2 days	Team B blocked 2 days; buddy pairing started
3	D3	API contract for incident ID agreed between teams (US-13 needs US-05)	Cross-team wait avoided with a mock
4	D4	BU SLA policy clarification obtained in the decision workshop	Team A blocked 2 days total (clarification before sprint) – reduced from Sprint 1
5	D6	Defect: retry created duplicate issue link (US-13)	Reopened once; idempotent key added
6	D8	Integration build breaks after merge; daily build catches it within the day	Fixed same day; 90% builds green
7	D10	US-06 not finished; moved to Sprint 3 (headroom used)	28 of 33 pts completed

Indicator (equivalent)	Original Sprint 1 (SIMULATED)	Revised Sprint 2 (SIMULATED, both teams)	Assessment
Completed vs planned	18/23 = 78%	28/33 = 85%	Improved; meets R2

#	Day	Simulated event (revised Sprint 2)	Effect
Blocked time	3 days	4 days total (A 2, B 2)	Within ≤ 4 per team; B's was setup
Avg cycle time	6.0 days	5.4 days	Improved
Average WIP per team	4.2 (limit 5)	3.8 (limit 5)	Under limit
Reopened stories	1/3 = 33%	1/5 = 20%	Better but above 15% target
Cross-team dependency wait	n/a	0.8 days average	Meets R3

E. Defend the decision

Aspect	Explanation
Benefits	All Must stories are kept, so the incident journey stays complete; extra capacity of the second team is used on a separate stream with limited shared code; scope cut is by simplification (configuration, templates), not by removing journey steps.
Limitations	Team B ramp-up is an assumption; reduced correlation (one rule) and deferred suggestions lower triage value; BU-specific logic is configuration only; coordination overhead (15%) reduces the benefit of 8 extra people.
Trade-offs	Breadth (suggestions, advanced correlation) is traded for completeness and predictability; documentation is lighter; Sprint 2 is deliberately tight, with a defined slip path.
Why feasible under the unchanged deadline	Committed 68 points vs ≈ 78 net capacity (87%). Sensitivity check: if Team B ramps at only 0.4 productivity in Sprint 2, capacity falls to about 76 points; the plan still fits (≈ 89% load). If a second person is lost, US-14 and N4/N5 are simplified first, as listed in the ROAM board.

Task 7 – Individual Technical Reflection

Most significant decision. The most significant decision was choosing a thin vertical slice (Option C) instead of building all eight modules. It forced me to define the incident journey first and use it to accept or defer every story. It also made the Sprint 1 and Task 6 simulations meaningful, because the same Must stories were preserved when capacity changed.

How evidence changed my strategy. My Sprint 1 prediction was that review/testing would be the bottleneck. The simulated indicators (3 blocked days, 1.5-day review wait and the RBAC dependency) showed that waiting for decisions and a large prerequisite story caused more delay. I therefore changed the Definition of Ready (decision log entry required), split prerequisite work and scheduled decision workshops a sprint ahead.

Where the design remains vulnerable. The design depends on assumptions (capacity factors, calibration, Team B ramp-up) with no real velocity history; the stakeholder responses and results are simulated; the 2 h/week stakeholder limit with four BU policy owners is a single point of failure; and keyword rules may not give good suggestions in real data. The 15% coordination factor is a guess.

Ethical concern addressed. I addressed privacy by using synthetic data only, restricting screens by role, and excluding personal data from logs and screenshots (DoD). I addressed transparency by labelling the rule-based suggestions as deterministic, requiring analyst confirmation, and clearly labelling every simulated result. For workload, WIP limits, a 10% buffer and a rule that unfinished work is re-planned rather than pushed through overtime protect the team. For accessibility, a WCAG-oriented check is part of the DoD.

What I would investigate next. Before using this with a real team I would: gather a real baseline of incident handling time and handovers; run a real Sprint 1 to replace the capacity guesses; test the Jira workflow with the real team for usability; validate the SLA matrix and approval rules with actual CAB and service-desk owners; and perform an actual security review of authorisation and the retry mechanism.

Sources and Tools Acknowledgement

- Schwaber, K. & Sutherland, J., The Scrum Guide (2020); Anderson, D., Kanban; Poppendieck, M. & T., Lean Software Development; Beck, K., Extreme Programming Explained.
- Scaled Agile Framework (SAFe) public documentation – concepts of PI planning, ART and Inspect & Adapt (adapted, not fully adopted).
- Atlassian Jira documentation (Scrum boards, workflows, reports); Trello, GitHub Projects and Azure DevOps product documentation (verify current features and prices).
- W3C Web Content Accessibility Guidelines (WCAG) 2.1.
- Tools: Jira (planned project management tool), Microsoft Word. An AI assistant (Claude) was used to help draft and structure this report; all stakeholder responses, event logs and indicators are simulated planning data, and I am responsible for reviewing and customising the content.

Appendix A – Jira Evidence Checklist and Capture Status

Ref.	Evidence required by Claude's checklist	Status
A1	Project settings / issue types / custom fields	Partial: project and backlog screenshots captured; settings/custom fields not shown.
A2	Backlog with five epics and fourteen stories	Partial: backlog/epic screenshots included; all priority and point values are not visible.
A3	Story acceptance criteria and dependencies	Not captured in provided screenshots.
A4	Workflow editor and WIP limits (5/3/2)	Not captured; screenshots show board columns only.
A5	Sprint 1 board, blocked flag and reopened bug	Included in Task 5 evidence.
A6	Sprint report, burndown, cumulative flow/control chart	Sprint Insights and burndown included; cumulative flow not captured.
A7	Team A/Team B board and revised Sprint 2/3	Not captured in provided screenshots.
A8	CSV issue export	Not captured in provided screenshots.

Evidence limitation: WIP limits, Team A/Team B board configuration and CSV export are not claimed as completed because the provided Jira screenshots do not show them. The sprint data and events are academic simulations.